

1 Introduction

The Time-Dependent Schrödinger equation is a partial differential equation that describes the evolution of wave functions Ψ of non-relativistic quantum systems over time. For two dimensional systems, this equation takes the form

$$(1) \quad i\hbar \frac{\partial \Psi(x, y, t)}{\partial t} = \left[-\frac{\hbar^2}{2m} \nabla^2 + V(x, y) \right] \Psi(x, y, t).$$

Some example quantum systems in 2D include a free particle in a box, a finite potential barrier, and a potential barrier with double slit. The latter of which is a variation of the famous experiment described by Thomas Young in 1801. It happens that these three examples provide suitable test cases for numerical solvers, which is discussed in section 3.

A common method of numerically solving the Schrödinger equation is the Finite Difference Method (FDM). FDM approximates the domain of the wave function by a grid of evenly spaced points and integrates over time in discrete time steps. While this method is powerful for solving such systems, the computational complexity increases drastically for larger system sizes. This is due to the fact that in a typical single threaded process, each grid point must be updated in series.

This algorithm is naturally extended to graphics processing units (GPUs), where it is possible to dispatch each grid point its own thread. This approach effectively speeds up the algorithm by integrating each point in parallel, while requiring certain considerations which are discussed in section 2.4.

2 Computational Methods

2.1 Wave Packet Generation

To discuss the behavior of any differential equation in the time domain, it is necessary to first define the conditions. We start with a mathematical description of a wave packet, representing a quantum mechanical particle.

In quantum mechanics, a localized particle can be prescribed by a linear superposition of plane waves. We represent the weighted sum of the waves by the integral

$$\Psi(x, y, 0) = A \iint_{-\infty}^{\infty} P(p_x, p_y) \exp \left[\frac{i}{\hbar} (p_x(x - x_0) + p_y(y - y_0)) \right] dp_x dp_y$$

where $P(p_x, p_y)$ is a gaussian distribution of momenta, given by

$$P(p_x, p_y) = B \exp \left[-\frac{(p_x - p_{x0})^2}{2\sigma_{p_x}^2} - \frac{(p_y - p_{y0})^2}{2\sigma_{p_y}^2} \right]$$

and A, B are normalization constants. This integral results the wave function

$$\begin{aligned} \Psi(x, y, 0) = C \exp \left[\frac{i}{\hbar} (p_{x0}(x - x_0) + p_{y0}(y - y_0)) \right] \\ \exp \left[-\frac{1}{2\hbar^2} (\sigma_{p_x}^2(x - x_0)^2 + \sigma_{p_y}^2(y - y_0)^2) \right] \end{aligned}$$

with the normalization constant C to be determined numerically.

We can then express the horizontal and vertical components of momentum by the relation

$$p_x = |p| \cos \alpha \quad p_y = |p| \sin \alpha$$

where α is the angle of the direction of propagation. Lastly, we make use of two physical laws before realizing the final equation. Assume the momentum distribution is rotationally symmetric, that is $\sigma_p := \sigma_{p_x} = \sigma_{p_y}$. Assume maximal simultaneous localization. We use Heisenberg's uncertainty principle to express σ_p in terms of σ_x .

$$\frac{\hbar}{2} = \Delta x \Delta y = \sigma_x \sigma_p \implies \sigma_p = \frac{\hbar}{2\sigma_x}$$

and the classical energy-momentum relation, where E is the non-relativistic kinetic energy of the particle

$$E = \frac{p^2}{2m} \implies p = \sqrt{2mE}$$

Finally, we have constructed the initial condition

$$\begin{aligned} \Psi(x, y, 0) = C \exp \left[i \frac{\sqrt{2mE_0}}{\hbar} [(x - x_0) \cos \alpha + (y - y_0) \sin \alpha] \right. \\ \left. - \frac{1}{4\sigma_x^2} [(x - x_0)^2 + (y - y_0)^2] \right]. \end{aligned}$$

2.2 Spatial Discretization

To approximate $\Psi(x, y)$ along its domain, we use a 2D Finite difference method. The domain is broken into an evenly spaced grid of values $\psi_{i,j} = \Psi(x_i, y_j)$ and $V_{i,j} = V(x_i, y_j)$, where $x_{i+1} - x_i = \Delta x$ and $y_{j+1} - y_j = \Delta y$. We take $\Delta x = \Delta y$.

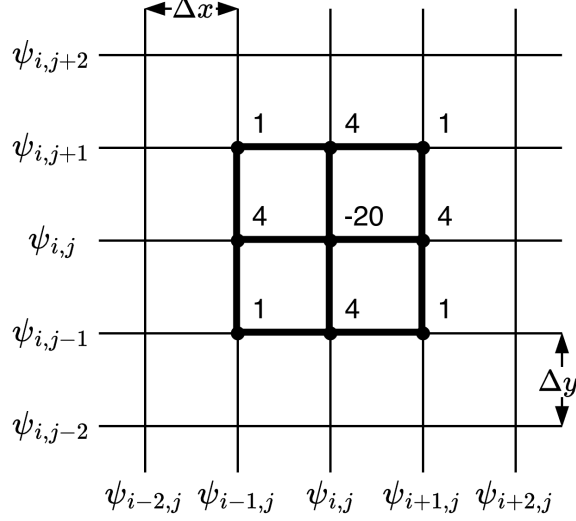


Figure 1: Graphical representation of the discretized domain, and ∇_9^2 operator.

It is also necessary to construct finite difference approximations of the Laplace operator, used in equation (1). We use the Nine-point stencil Laplace operator ∇_9^2 for its greater isotropy over the central difference kernel. This, in theory, may lead to improved accuracy when computing non axis-aligned wave modes.

$$\nabla_9^2 \psi_{i,j} = \frac{1}{6\Delta x^2} [\psi_{i-1,j+1} + 4\psi_{i,j+1} + \psi_{i+1,j+1} + 4\psi_{i-1,j} - 20\psi_{i,j} + 4\psi_{i+1,j} + \psi_{i-1,j-1} + 4\psi_{i,j-1} + \psi_{i+1,j-1}]$$

Lastly, we apply this discretization to the 2D Schrödinger equation (1).

$$(2) \quad i\hbar \frac{\partial \psi_{i,j}}{\partial t} = \left[-\frac{\hbar^2}{2m} \nabla_9^2 + V_{i,j} \right] \psi_{i,j}.$$

It is also important to discuss boundary conditions. We apply Dirichlet boundary conditions, setting $\psi = 0$ on the boundary of the discretized domain.

2.3 Time Integration

Since we have discretized the spatial domain, it is now necessary to discuss how we will evolve the system over time. We can see that discretization allows us to treat the Schrödinger partial differential equation as a system of coupled ordinary differential equations. Thus a natural choice of time integrator is a Runge-Kutta method. We use a second order, explicit Runge-Kutta method (RK2). This method is described by a two stage process

$$(3) \quad \begin{cases} \tilde{\psi}_{k+1} = \psi_k + (\Delta t)S(t_k, \psi_k) \\ \psi_{k+1} = \psi_k + \frac{\Delta t}{2} \left[S(t_k, \psi_k) + S(t_{k+1}, \tilde{\psi}_{k+1}) \right] \end{cases}$$

where $t_{k+1} - t = \Delta t$. We must also write equation (2) in terms of the time derivative.

$$(4) \quad S := \frac{\partial \psi_{i,j}}{\partial t} = -\frac{i}{\hbar} \left[-\frac{\hbar^2}{2m} \nabla_9^2 + V_{i,j} \right] \psi_{i,j}.$$

While RK2 does not inherently conserve certain quantities, such as energy and momentum, we can choose a small enough time step such that these effects are negligible. This is shown in section 3 for the wave function norm.

2.4 Algorithmic Implementation

As previously stated, the GPU allows for the execution of code in parallel at vastly larger scales than the CPU. This is achieved through the process of single instruction, multiple data (SIMD). We dispatch one GPU thread per grid point. Whereas a typical CPU may support on the order of 10 hardware threads, a modern GPU can execute tens of thousands of threads concurrently.

This parallelism introduces additional considerations, most notably thread safety. A race condition can arise when multiple threads simultaneously read from or write to the same memory location. To avoid this, we implement a ping-pong buffer scheme, described as follows.

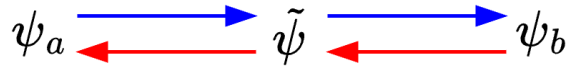


Figure 2: Diagram of ping pong buffer scheme

We begin with the initial data stored in the buffer ψ_a . Stage 1 of the RK2 method is evaluated into the temporary buffer $\tilde{\psi}$, after which Stage 2 is evaluated

into ψ_b , completing one time-integration step. The next step reverses the bindings of the buffers: Stage 1 is evaluated from ψ_b into $\tilde{\psi}$, and Stage 2 updates ψ_a . This alternating (‘ping-pong’) pattern continues at each time iteration, with all grid points updated in parallel, and ensures no two threads write and access the same memory location simultaneously.

3 Simulation Output

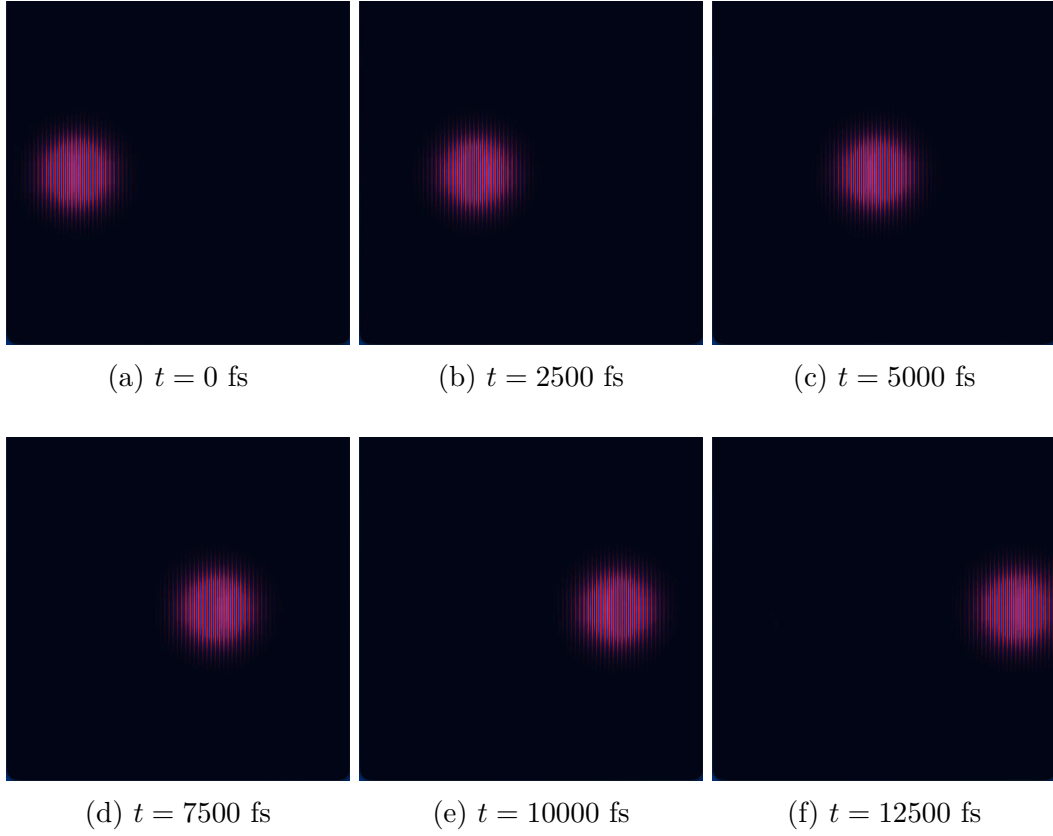


Figure 3: Simulation of a free electron wave packet. $E_0 = 0.01$ eV, $\alpha = 0$ rad, $\sigma_x = 50$ nm, $\Delta x = 1$ nm, $L = 1000$ nm, $\Delta t = 1$ fs. Real part of wave function, red is positive, blue is negative.

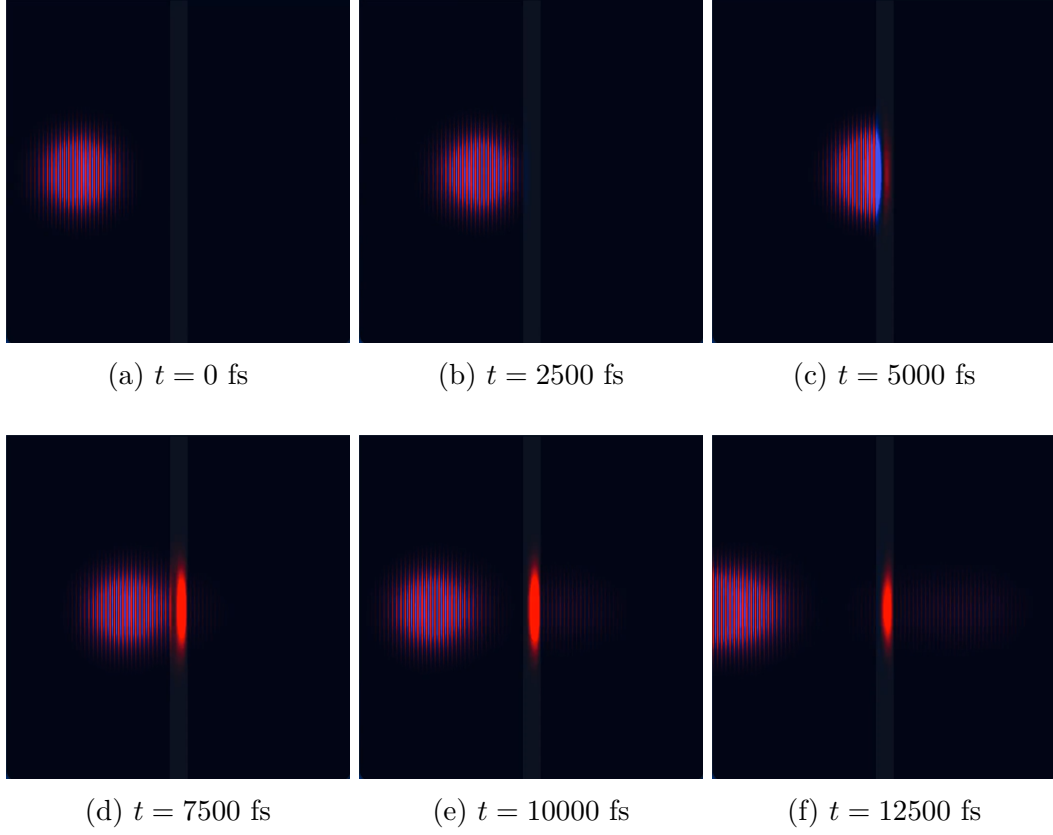


Figure 4: Simulation of an electron wave packet interacting with a potential barrier. $E_0 = 0.01$ eV, $\alpha = 0$ rad, $\sigma_x = 50$ nm, $\Delta x = 1$ nm, $L = 1000$ nm, $\Delta t = 1$ fs, $V_{barrier} = 0.01$ eV, $L_{barrier} = 50$ nm. Real part of wave function, red is positive, blue is negative.

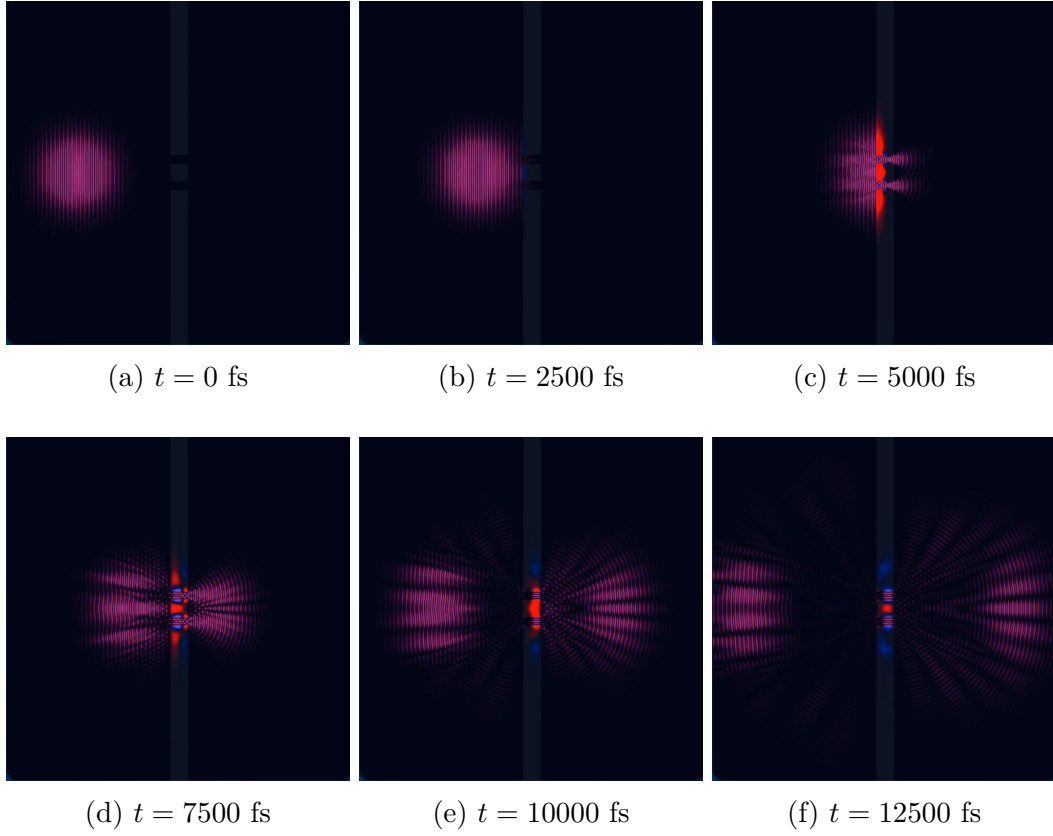


Figure 5: Simulation of an electron wave packet interacting with a double slit barrier. $E_0 = 0.01$ eV, $\alpha = 0$ rad, $\sigma_x = 50$ nm, $\Delta x = 1$ nm, $L = 1000$ nm, $\Delta t = 1$ fs, $V_{\text{barrier}} = 0.01$ eV, $L_{\text{barrier}} = 50$ nm, $d = 50$ nm, $a = 25$ nm. Real part of wave function, red is positive, blue is negative.

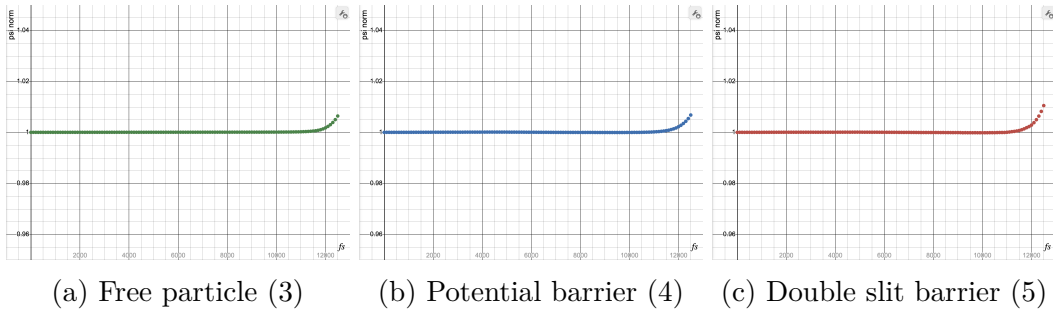


Figure 6: Wave function norms $|\Psi|^2$ over time (fs), corresponding to figures 3, 4, 5.

4 Discussion

The simulations reproduce the expected quantum-mechanical behavior across all three scenarios. In the free-particle test case, the wave packet propagates across the domain while gradually dispersing, consistent with theoretical predictions. This validates the correct implementation of the Laplacian operator in equation (4). For the second test case, a particle interacting with a finite barrier, the packet splits into reflected and transmitted components, demonstrating partial tunneling. This is expected when the particle's kinetic energy is comparable to the barrier height. This confirms that the potential term of (4) is working correctly. For the experimental case, the finite double-slit barrier, the simulation produces a clear interference pattern. This is the expected result, and shows the system's capability of resolving small details.

Across all simulations, the wave function norm remains nearly constant for a time, indicating that the explicit RK2 integrator, under a sufficiently small time step, maintains numerical stability for short simulations. However, due to the non unitary nature of RK2, the solution eventually begins to blow up. The simulation length is a potential drawback to this method, however, it can be improved by taking a smaller time step.

Overall, the method provides reliable and physically consistent results while taking advantage of speedups coming from the GPU-parallized finite-difference method. This method produces accurate solutions to the 2D time-dependent Schrödinger equation.

5 References

- [1] J. Bernstein, P. M. Fishbane, and S. Gasiorowicz, *Modern Physics*. Upper Saddle River, NJ: Prentice Hall, 2000.
- [2] J. Tobochnik, H. Gould, and C. Wolfgang, *An Introduction to Computer Simulation Methods: Applications To Physical Systems*, 3rd ed. Open Source Physics
- [3] “Vulkan documentation,” Vulkan Documentation Project, <https://docs.vulkan.org/spec/latest/> (accessed Nov. 24, 2025).
- [4] D. Swigon, Honors Ordinary Differential Equations Lecture Notes, Apr. 14, 2025